Hawaii Manta Tracker Automated Ventral Photo-ID Matcher

Technical status brief for AI / computer-vision collaboration

Objective

We are building a local, deterministic, explainable photo-identification pipeline for reef manta rays. A new ventral manta image should return catalog candidates sorted by likely identity, ideally placing good-quality true resights in the top 10 and lower-quality photos in the top 20-50 for admin review.

The biological signal is ventral pigment: chest/gill-adjacent markings, central/navel region, and pelvic/clasper region. Wing-margin spots are secondary. Matches should be explainable by visible pigment regions and anatomical consistency, not generic image similarity.

Current Architecture

Layer	Current implementation
Core matcher	Python/OpenCV in scripts/matching/pigment_region_matcher.py
Worker API	scripts/matching/matcher_api_server.py at http://127.0.0.1:8766
App UI	React/Vite admin pages and Add Sighting match modal
Caching	Per-photo JSON signatures in scripts/matching/cache/photo_signatures/
Anchors	Best catalog ventral photos plus optional best-manta-ventral resight anchors

Algorithm Pipeline

- Normalize image and generate body / valid-photo mask, including special handling for white-background cutouts and rotated image padding.
- Use widened central ventral ROI; narrow gill-only ROIs caused missed pigment and false zone comparisons.
- Enhance dark pigment with local contrast / spotness maps, then segment connected pigment regions inside body + ROI.
- Describe each region with centroid, normalized area, contour/shape, darkness/contrast, anatomical zone, and local neighbor-constellation features.
- Prefilter likely anchors, exact-score a smaller set, aggregate by catalog ID, and return ranked candidates plus debug overlays.

What Changed From Earlier Attempts

Earlier edge/blob/keypoint approaches detected halos, body margins, gill slits, cephalic fin edges, water texture, reef/sand, and compression noise. They produced many false anchors. The current approach favors fewer biologically meaningful connected pigment regions and includes penalties/diagnostics for weak zone coverage, large-region mismatch, margin artifacts, and unstable local geometry.

Measured Performance To Date

Benchmark	Rank #1	Top 10	Top 20	Top 50	Interpretation
Exact-image self-match	792/792	792/792	792/792	-	Sanity check passes; not proof of true resight accuracy.
Controlled 10-query neighbor-constellation	3/10	7/10	8/10	10/10	Promising admin-assist behavior on a small curated set.
Stratified best-manta-ventral 10-query run	4/10	5/10	5/10	6/10	Mixed; not production-grade autonomous ID.

Status: useful as an experimental admin sorter in some cases, but the correct catalog may still rank outside the top 20 or top 50 for difficult images. Example observed case: Catalog 4 / Photo 3800 expected catalog ranked around #38 in UI testing.

Known Failure Modes

- Body segmentation remains fragile for some rotated, cropped, or backlit photos; straight padding edges can pollute masks.
- ROI can be anatomically wrong when the manta is rotated or strongly parallaxed; tilted chest/wing/pelvic regions get compared incorrectly.
- Detector can still treat gill slits, mouth/cephalic edges, shadows, and contrast halos as pigment.
- Large obvious pigment patches are not always weighted strongly enough to reject false candidates.
- Dark/backlit photos can confuse shaded white body with true dark pigment unless quality/contrast handling improves.
- Export/storage drift can affect benchmarks; one manifest referenced a missing local file while the same photo existed under another export name.

Current UI Integration

The Add Sighting “Find Catalog Match” modal includes an experimental score-order option. Admins can still browse by Catalog ID and filters, but can run the local matcher to reorder candidates. The UI labels suggestions as experimental and displays progress for anchor loading and candidate scoring. A separate /admin/matching page lets us test a known query image and inspect top candidates, score components, and debug overlays.

Deployment / Operational Status

The React frontend is Vite/Vercel-compatible. The matcher is currently a local Python worker and should not be placed in a browser or Supabase Edge Function. A live deployment would need a persistent Python/OpenCV service with a warmed anchor cache and a configured VITE_MATCHER_API_BASE endpoint. Locally, the desktop launcher starts both the app and matcher worker.

Next Technical Improvements

- Build a consolidated precomputed anchor index so startup and repeated matching are faster.
- Improve subject segmentation, especially rotated/cropped images, padding boundaries, and water/reef backgrounds.
- Model anatomical zones more explicitly: chest/gill, central/navel, pelvic, and optional wing-secondary regions.
- Learn stable same-animal pigment priors from multi-resight catalogs; use unstable features as negative evidence.
- Improve parallax/rotation robustness using local constellation descriptors and zone-wise comparisons rather than absolute XY alone.
- Add quality classification so poor photos are flagged low-confidence instead of expected to rank top 10.
- Build database/photo QC so rows, storage objects, exports, manifests, and cached signatures stay consistent.

Where An AI/CV Expert Could Help Most

- Robust manta body segmentation from water/reef/background and rotated padding.
- A biologically constrained pigment-region descriptor tolerant of parallax and partial visibility.
- A two-stage retrieval design: fast global/region prefilter followed by slower explainable verification.
- A scoring/ranking model combining deterministic region features with learned priors while staying inspectable.
- Benchmark design with curated positives/negatives, multi-anchor catalog scoring, and quality-tier metrics.

Bottom line: the project has moved from noisy blob/keypoint matching toward explainable pigment-region matching with UI integration and evaluation harnesses. The system is already useful for experimentation and occasional browsing reduction, but segmentation, anatomical zone consistency, parallax robustness, speed, and data QC are the main blockers before it can be considered reliable production ID support.